Project Title

SecureAPI Guard

contents

Project Title 1
Abstract 5
Keywords 6
How to Use This Document 6
1. Introduction 6
1.1 Background 6
1.2 Project Overview 7
1.3 Motivation 7
1.4 Report Organization 7
2. Problem Statement and Objectives 8
2.1 Problem Statement 8
2.2 General Objective 8
2.3 Specific Objectives 8
2.4 Expected Outcomes 9
3. Scope and Limitations 9
3.1 Included Scope 9
Request metadata and request identification where implemented. 9
3.2 Out of Scope for the Current Prototype 9
3.3 Known Limitations 10
3.4 Assumptions 10
4. Background and Security Concepts 10
4.1 API 10
4.2 Middleware 10
4.3 SQL Injection 11
4.4 Cross-Site Scripting (XSS) 11
4.5 Detection, Logging, and Blocking 11
4.6 Rate Limiting 12
5. Requirements Analysis 12
5.1 Functional Requirements 12

5.2 Non-Functional Requirements 13
5.3 Users and Roles 14
6. System Architecture and Design 14
6.1 High-Level Architecture 14
6.2 Main Components 15
6.3 Request Lifecycle 16
6.4 Data Flow 17
6.5 Suggested Future Customer Gateway 17
7. Implementation and Technical Details 17
7.1 Technology Stack 17
7.2 Detection Approach 18
7.3 Error Handling and Reliability 18
7.4 Frontend Behavior 18
7.5 Configuration and Secrets 19
8. Installation, Configuration, and Operation 19
8.1 Prerequisites 19
8.2 Install Dependencies 19
8.3 Run Automated Tests 19
8.4 Start the Server 20
8.5 Stop the Server 20
8.6 Basic Demonstration Procedure 20
8.7 Troubleshooting 20
9. Testing and Evaluation 21
9.1 Testing Strategy 21
9.2 Previously Recorded Automated Test Result 22
9.3 Synthetic Detection Evaluation 22
9.4 Local Performance Measurement 23
9.5 Test Case Matrix 23
9.6 Evaluation Limitations 24
10. Security, Privacy, and Reliability Considerations 25
10.1 Defense in Depth 25
10.2 Data Minimization 25
10.3 Multi-Customer Isolation 25
10.4 Safe Failure 25
10.5 Public Deployment Readiness 25
11. Results and Discussion 26

11.1 Achievements 26
11.2 Interpretation 26
11.3 Project Status Statement 26
12. Conclusion and Future Work 27
12.1 Conclusion 27
12.2 Future Work 27
References 28
Appendix A — Demonstration Scenarios A.1 Normal Request 28
A.2 Suspicious Pattern 29
A.3 Error Resilience 29
A.4 Dashboard Verification 29
Final Submission Note 31

Field	Description
Project title	SecureAPI Guard
Document type	Technical documentation and project report
Project category	Cybersecurity / API security proof of concept
Implementation stack	Node.js, Express, HTML, CSS, vanilla JavaScript, automated Node.js tests
Document status	Submission draft — requires student review and institution-specific formatting

Abstract

Application Programming Interfaces (APIs) enable websites, mobile applications, and services to exchange data. Because APIs accept externally supplied requests, they can be exposed to suspicious input, automated abuse, and application-layer attacks. SecureAPI Guard is an educational proof of concept designed to inspect API requests for selected suspicious patterns, including patterns associated with SQL injection and Cross-Site Scripting (XSS), and to present relevant security activity through a web-based interface.

The system is implemented using Node.js and Express, with a browser-based frontend built from HTML, CSS, and vanilla JavaScript. Its logical components include HTTP routes, security-inspection middleware, request metadata, rate limiting, metrics

services, and automated tests. The project demonstrates how request inspection, event recording, and dashboard reporting can be organized in a small application.

The project is a prototype rather than a production-grade Web Application Firewall (WAF). Its detection is rule-based and heuristic; therefore, it may produce false positives and false negatives. Previous development notes recorded 13 passing automated tests and an evaluation on a fixed synthetic dataset, but these results must be rerun against the final submitted code. The report distinguishes measured prototype results from features that require further engineering, including multi-customer authentication, durable event storage, hardened gateway deployment, and production recovery controls.

Keywords

API security; Express.js; Node.js; SQL injection; Cross-Site Scripting; request inspection; security monitoring; rate limiting; proof of concept.

How to Use This Document

This document combines the project report and technical documentation. Before submission, replace bracketed placeholders, verify the final source code, rerun tests, update results, and ensure that every claimed feature is demonstrably implemented. Institution-specific cover-page rules, declaration pages, supervisor signatures, and citation styles should be added where required.

1. Introduction

1.1 Background

Modern applications commonly expose APIs to accept login requests, retrieve information, update records, and connect frontend applications with backend services. An API endpoint may receive data from browsers, mobile applications, partner

systems, or automated clients. This makes request validation and monitoring important parts of application security.

SQL injection occurs when untrusted input is incorrectly incorporated into SQL commands. Cross-Site Scripting occurs when untrusted content is interpreted as executable script in a browser context. Both risks are best addressed through secure application design—for example, parameterized database queries and context-aware output encoding. Request inspection can provide an additional monitoring layer, but it cannot replace these controls.

1.2 Project Overview

SecureAPI Guard is a small, rule-based API security inspection demonstration. It provides an Express backend and browser-based pages for presenting information and exercising selected request-inspection behavior. The project is intended to demonstrate a practical security workflow: receive a request, inspect it using configured rules, record relevant metadata or events, return an HTTP response, and display supported metrics.

1.3 Motivation

For students and developers, API security can be difficult to understand without a working demonstration. SecureAPI Guard aims to make selected request-inspection concepts observable and testable using a controlled local environment. It also provides a foundation for studying error handling, request metadata, rate limiting, automated testing, and dashboard design.

1.4 Report Organization

The report describes the problem and objectives, defines the prototype's scope, explains the architecture and implementation, provides operational instructions,

discusses evaluation evidence, and identifies the work needed before a real customer deployment.

2. Problem Statement and Objectives

2.1 Problem Statement

Developers need ways to inspect suspicious API requests and review security-related activity. A demonstration that only shows a landing page or static dashboard does not establish whether backend requests are processed, how suspicious patterns are detected, or what evidence is available after a request. The project addresses this learning and demonstration need by implementing a small request-inspection workflow with testable backend behavior and visible reporting.

2.2 General Objective

To design, implement, and evaluate an educational API security inspection and monitoring prototype that identifies selected suspicious request patterns and exposes supported security information through a web interface.

2.3 Specific Objectives

- Implement an HTTP backend using Node.js and Express.
- Inspect selected request inputs for patterns associated with SQL injection and XSS.
- Use middleware and routes to separate request handling from security inspection.
- Apply configured rate limiting and safe request metadata handling.
- Provide a browser-based interface for interacting with the demonstration and reviewing supported metrics.
- Create automated tests for key API and detection behaviors.
- Document architecture, setup, evaluation, limitations, and possible future improvements.

2.4 Expected Outcomes

The expected outcome is a runnable proof of concept that can be demonstrated locally, tested using controlled inputs, and used to explain the difference between detection, logging, and enforcement. The project is not intended to prove that all API attacks can be detected or prevented.

3. Scope and Limitations

3.1 Included Scope

- Node.js and Express backend application.
- Rule-based inspection for selected SQL injection and XSS patterns.

Request metadata and request identification where implemented.

- Rate limiting and security-related middleware.
- API routes for health, testing, dashboard metrics, and demonstration functions, subject to the final source-code audit.
- HTML, CSS, and vanilla JavaScript pages.
- Automated tests using Node.js testing tools.

3.2 Out of Scope for the Current Prototype

- Guaranteed detection or prevention of every attack.
- A fully hardened, high-availability commercial WAF.
- Production-grade multi-tenant identity, billing, and customer administration unless separately implemented and verified.
- Tamper-proof audit storage or a formally managed security event retention system.
- A guarantee that every customer's external API is protected merely by registering it in the dashboard.

- Claims of machine-learning-based detection; the described detection approach is rule-based.

3.3 Known Limitations

The detector relies on configured patterns and heuristics. Attack techniques can vary in encoding, context, and structure; benign inputs may resemble suspicious patterns. The dashboard's authentication and customer separation must be verified before any multi-user deployment. Local or in-memory logs and metrics may not survive process restarts and should not be described as durable audit records unless persistent storage is implemented. A local performance benchmark does not establish internet-scale throughput, availability, or production security.

3.4 Assumptions

The prototype is evaluated in a controlled development environment, with requests generated by the developer or authorized testers. Any external API integration must be explicitly configured and tested. No third-party target should be scanned or tested without authorization.

4. Background and Security Concepts

4.1 API

An Application Programming Interface (API) defines how one software component requests data or actions from another. In a web application, an API commonly receives HTTP requests and returns HTTP responses, often encoded as JSON.

4.2 Middleware

Express middleware is a function that runs during the request-response process. Middleware can inspect request properties, attach metadata, enforce limits, or pass

control to the next middleware or route. Error-handling middleware receives errors and produces a controlled response.

4.3 SQL Injection

SQL injection is a vulnerability that may arise when applications construct SQL statements unsafely using untrusted input. Parameterized queries and safe data-access patterns are primary defenses. A pattern detector can flag inputs that resemble common injection attempts, but a pattern match alone does not prove that a vulnerability exists.

4.4 Cross-Site Scripting (XSS)

XSS occurs when untrusted content is handled in a way that causes a browser to execute unintended script. Context-aware output encoding, safe DOM APIs, and appropriate content security policies are important defenses. A request inspector may identify selected suspicious strings, but it cannot determine every browser rendering context.

4.5 Detection, Logging, and Blocking

Term	Meaning
Detection	A rule identifies a request as matching a suspicious pattern.
Logging	The system records an event or metadata about the request.
Monitoring mode	The request may continue while suspicious activity is recorded.
Blocking mode	The system rejects a request before it reaches the protected application, if this behavior is implemented and correctly positioned in the request path.

These outcomes must be reported separately. A logged detection is not automatically a blocked attack, and a blocked request is not proof that an attack would otherwise have succeeded.

4.6 Rate Limiting

Rate limiting restricts how many requests a client can make during a configured period. It can reduce some forms of automated abuse, but it does not replace authentication, authorization, input validation, or capacity planning.

5. Requirements Analysis

5.1 Functional Requirements

ID	Requirement	Verification approach
FR-01	The server starts using the documented command.	Start the application and request its local URL.
FR-02	The backend accepts supported HTTP requests.	Exercise existing API routes.
FR-03	Selected suspicious patterns can be detected.	Run controlled detection test cases.
FR-04	Request processing returns controlled errors for invalid input.	Send malformed and invalid requests.
FR-05	Security-related metadata or events are recorded where implemented.	Inspect the relevant event/metrics response.
FR-06	The dashboard presents supported backend data.	Compare displayed metrics with backend

		results.
FR-07	Rate limiting is applied according to configuration.	Send requests above the configured limit in a controlled test.
FR-08	Automated tests verify key behaviors.	Run `npm.cmd test` and inspect the output.

5.2 Non-Functional Requirements

ID	Requirement	Importance
NFR-01	Reliability: one ordinary request failure should not crash the whole process.	High
NFR-02	Security: avoid exposing secrets, sensitive request content, or stack traces.	High
NFR-03	Maintainability: keep routes, middleware, metrics, and utilities separated.	High
NFR-04	Usability: provide clear navigation, understandable statuses, and useful errors.	Medium
NFR-05	Testability: provide repeatable tests and clearly defined test data.	High
NFR-06	Responsive presentation:	Medium

support common desktop and mobile screen sizes.

5.3 Users and Roles

In the current prototype, the primary user is the developer or evaluator operating the demonstration. A future customer-facing version would need authenticated roles and strict customer data isolation. Do not represent future roles as implemented until verified.

6. System Architecture and Design

6.1 High-Level Architecture



```
├── Security inspection logic
├── Metrics and event handling
└── Centralized error handling (verify current implementation)
        |
        ▼
HTTP response + recorded metrics/events (as implemented)
```

This diagram shows the logical structure. The final diagram must be checked against the actual source code, especially the exact middleware order, data storage, and route paths.

6.2 Main Components

Component	Responsibility
`server.js`	Application entry point and server startup, according to current project conventions.
`src/app.js`	Express application setup, middleware registration, and route mounting where implemented.
`src/middleware/securityInspection.js`	Request inspection middleware; verify exact decision and error behavior in source.
`src/middleware/rateLimiter.js`	Rate-limiting configuration and enforcement.
`src/routes/healthRoutes.js`	Health-related endpoint(s).
`src/routes/testRoutes.js`	Endpoints used for security testing.
`src/routes/dashboardRoutes.js`	Dashboard data endpoints.
`src/routes/demoRoutes.js`	Demonstration endpoints, if present in

	the final version.
`src/services/metricsService.js`	Metric collection or calculation.
`src/utils/requestId.js`	Request ID generation or handling.
`src/utils/requestMetadata.js`	Request metadata extraction or formatting.
`public/`	Static frontend HTML, CSS, and JavaScript.
`tests/`	Automated API, detection, and performance tests.

6.3 Request Lifecycle

1. A client sends an HTTP request to the local Express server.
2. Configured middleware processes the request in registration order.
3. Request identification and metadata are attached if the relevant middleware is enabled.
4. Security inspection evaluates the request against configured rules.
5. The route or middleware determines the response according to implemented behavior.
6. Metrics or security events are updated where applicable.
7. The server returns an HTTP response to the client.
8. The frontend displays the response or fetches supported dashboard data.

The exact order matters. For example, if inspection is intended to protect a route, it must run before the protected handler processes the request. The source code and tests should verify this behavior.

6.4 Data Flow

The frontend communicates with the backend using HTTP. Backend routes process requests and return data. The metrics service aggregates supported values for dashboard presentation. The current persistence strategy must be verified: if records are held only in memory, they can be lost on restart; if files are used, file permissions, sanitization, rotation, and retention need attention.

6.5 Suggested Future Customer Gateway

A future deployment that protects a separate company's API would need an explicitly configured gateway or installable middleware. The intended gateway flow would be: customer application → SecureAPI Guard inspection layer → customer's backend. Registering an API in a user interface alone does not route traffic through the inspection layer. A gateway requires secure upstream configuration, TLS, timeouts, destination validation, customer authentication, and tested enforcement behavior.

7. Implementation and Technical Details

7.1 Technology Stack

Technology	Purpose
Node.js	JavaScript runtime for the backend.
Express	HTTP application framework and routing.
Helmet	Security-related HTTP response headers, subject to configured options.
express-rate-limit	Rate limiting for configured routes.
HTML	Page structure.
CSS	Visual design and responsive layout.
Vanilla JavaScript	Browser-side interaction and API

	requests.
Node.js test runner	Automated tests using the `node --test` command.

7.2 Detection Approach

The prototype uses rule-based or heuristic inspection rather than a machine-learning classifier. Rules inspect selected request components for patterns associated with SQL injection or XSS. The detector's exact inputs, transformations, pattern list, and action must be documented from the final source code. Do not assume all HTTP fields are inspected unless the code verifies that claim.

A robust evaluation should include benign inputs, known suspicious patterns, encoded variations, boundary cases, and false-positive examples. Results should report the dataset size and composition, not only an overall percentage.

7.3 Error Handling and Reliability

A reliable Express service should validate input, use consistent error responses, catch asynchronous failures, apply request and upstream timeouts where relevant, and avoid returning stack traces to clients. A fatal process-level exception may require a controlled shutdown and supervisor restart rather than attempting to continue in an unknown state. These controls should be confirmed in the current implementation before being described as completed.

7.4 Frontend Behavior

The frontend should treat backend responses as untrusted input, safely render event details, check HTTP status codes, handle invalid JSON and network failures, and distinguish a failed request from a valid empty result. Dashboard numbers should be

fetched from the backend and must not be replaced with hard-coded values when a request fails.

7.5 Configuration and Secrets

Configuration should be provided through environment variables where appropriate. The `.env.example` file should contain placeholders only. Actual `.env` files, credentials, private keys, access tokens, and customer data must not be committed to source control. A filename scan is not sufficient to prove that a repository contains no secrets; file contents and Git history must also be reviewed.

8. Installation, Configuration, and Operation

8.1 Prerequisites

- Windows with PowerShell or another supported terminal.
- A compatible Node.js installation and npm.
- The project repository or project folder.
- Internet access only when installing dependencies, if they are not already installed.

8.2 Install Dependencies

Open the project folder in VS Code. In the integrated terminal, verify that the current directory is the project root containing `package.json`. On the user's Windows PowerShell setup, `npm.cmd` can be used if PowerShell blocks the `npm.ps1` script.

```
npm.cmd install
```

8.3 Run Automated Tests

```
npm.cmd test
```

The test command should complete and report pass/fail results. If a test fails, preserve the full error output and investigate it rather than assuming the application is ready.

8.4 Start the Server

```
npm.cmd start
```

Keep the terminal open while the server is running. Read the startup message to confirm the actual port and address. If the application uses port 3000, open the following address in a browser:

```
http://localhost:3000/
```

If the port differs, use the address printed by the server. `localhost` means the server is available on the same computer; it does not make the service publicly available on the internet.

8.5 Stop the Server

Return to the terminal running the server and press Ctrl+C. This requests the process to stop. If the process does not stop cleanly, inspect its logs and shutdown behavior.

8.6 Basic Demonstration Procedure

1. Start the server and confirm the expected local address.
2. Open the landing page.
3. Open the testing interface using the site's navigation.
4. Submit a benign test input and observe the response.
5. Use a controlled suspicious test case provided by the project.
6. Record the response status and detection result.
7. Open the dashboard and compare its metrics with the test activity.
8. Run the automated tests and preserve the output for the final report.

8.7 Troubleshooting

Problem	Possible cause	Action
`npm` is not recognized or	Shell execution policy or	Try `npm.cmd` and verify

PowerShell blocks it	command resolution.	Node.js installation.
`Cannot find module ...server.js`	Terminal is in the wrong directory or the entry point differs.	Change to the project root and check `package.json` scripts.
Port already in use	Another process uses the configured port.	Stop the other process or configure a free port safely.
Dashboard does not load data	Backend endpoint failure, incorrect path, or browser request error.	Inspect server output and browser developer tools; verify the endpoint.
Tests fail	Regression, missing dependency, or environment mismatch.	Run the test command again and inspect the first failing assertion.
Server exits unexpectedly	Unhandled error, configuration problem, or dependency failure.	Inspect logs and reproduce the issue with a controlled test.

9. Testing and Evaluation

9.1 Testing Strategy

Testing should cover individual detection rules, HTTP endpoints, integration behavior, error handling, and the frontend's handling of backend responses. Automated tests should be repeatable and should use controlled test data. Performance measurements should be made under documented local conditions and should not be generalized to production traffic.

9.2 Previously Recorded Automated Test Result

Development notes from an earlier project state recorded 13 automated tests passed, 0 failed, and 0 skipped. This is historical evidence, not confirmation of the final submission state. Rerun `npm.cmd test` against the exact version to be submitted and replace the table below with the current terminal output.

Measure	Previously recorded value	Interpretation
Tests passed	13	Previously recorded; must be rerun.
Tests failed	0	Previously recorded; must be rerun.
Tests skipped	0	Previously recorded; must be rerun.

9.3 Synthetic Detection Evaluation

A previous fixed synthetic evaluation recorded 12 cases: 7 true positives (TP), 1 false positive (FP), 4 true negatives (TN), and 0 false negatives (FN). The dataset was small and synthetic. These values describe only that specific test set and must not be presented as real-world detection performance.

Metric	Calculation	Previously recorded result
Recall	TP / (TP + FN) = 7 / 7	1.00 (100%)
Precision	TP / (TP + FP) = 7 / 8	0.875 (87.5%)
False-positive rate	FP / (FP + TN) = 1 / 5	0.20 (20%)
Accuracy	(TP + TN) / total = 11 / 12	0.9167 (about 91.7%)

The sample size is too small to support broad claims. A recall of 100% on seven positive examples does not mean the detector will identify every real attack. The 20% false-positive rate on this tiny set also indicates that benign inputs may be flagged.

9.4 Local Performance Measurement

An earlier localhost benchmark recorded 50 synthetic HTTP POST requests to `/api/test/inspect`, with a median latency of 5.49 milliseconds and a 95th-percentile latency (p95) of 21.75 milliseconds. These figures are historical and environment-specific. Rerun the benchmark against the final build and document the machine, runtime version, test method, warm-up, concurrency, and whether the measurement includes network or browser overhead.

Benchmark item	Previously recorded result	Caution
Requests	50 synthetic requests	Small sample.
Median latency	5.49 ms	Localhost measurement; not a production SLA.
p95 latency	21.75 ms	Sensitive to environment and test conditions.

9.5 Test Case Matrix

Test ID	Input / condition	Expected behavior	Actual result
TC-01	Valid benign request	Appropriate response; no unexpected	[Fill after rerun]

		detection.	
TC-02	Known SQLi-like test input	Expected configured detection outcome.	[Fill after rerun]
TC-03	Known XSS-like test input	Expected configured detection outcome.	[Fill after rerun]
TC-04	Malformed JSON	Controlled client error; server remains available.	[Fill after rerun]
TC-05	Missing required field	Validation response where applicable.	[Fill after rerun]
TC-06	Rate-limit threshold exceeded	Configured rate-limit response.	[Fill after rerun]
TC-07	Unknown route	404 response; process remains available.	[Fill after rerun]
TC-08	Backend endpoint failure	Clear frontend error state; no fake metrics.	[Fill after rerun]

9.6 Evaluation Limitations

Evaluation quality depends on the test dataset, ground-truth labels, route coverage, environment, and repeatability. The current evidence does not establish resistance to evasion, security under adversarial traffic, availability under load, or safe deployment in front of a production API. Those require additional testing and review.

10. Security, Privacy, and Reliability Considerations

10.1 Defense in Depth

Request inspection is only one layer of a secure API design. The protected application should still use authentication, authorization, parameterized queries, safe output encoding, secure session management, least privilege, and appropriate business-rule validation.

10.2 Data Minimization

Collect only the event data needed for investigation. Avoid logging secrets and sensitive bodies. Restrict access to logs and metrics, define retention periods, and protect stored data with suitable file permissions or database access controls.

10.3 Multi-Customer Isolation

A public customer-facing version must authenticate users and scope every API, event, and metric query to the authorized customer. Client-side filtering alone is not access control. This must be enforced on the server and tested with multiple test accounts before offering the service to separate organizations.

10.4 Safe Failure

A security service must not silently disable inspection after an error. Define whether each failure should reject the request or permit it, based on the security requirements and deployment mode. For a protective gateway, fail-open and fail-closed choices have different availability and security consequences; they must be explicit, documented, and tested.

10.5 Public Deployment Readiness

Before deployment, review authentication, authorization, TLS, secrets, dependency updates, request limits, upstream validation, SSRF protections where relevant,

persistent storage, backups, monitoring, rate limiting, denial-of-service considerations, and recovery procedures. Do not expose the development server directly to the public internet simply to make the dashboard accessible.

11. Results and Discussion

11.1 Achievements

The project establishes a foundation for demonstrating API request inspection through an Express application. Its modular structure separates important concerns such as routes, middleware, request metadata, metrics, frontend interaction, and tests. The previously recorded synthetic evaluation and localhost benchmark provide an initial testing baseline, provided they are rerun and confirmed for the final code.

11.2 Interpretation

The prototype can support a useful educational demonstration of how suspicious patterns are identified and how request events can be surfaced. However, successful detection on a limited set of inputs does not establish that the system is secure against varied real-world attacks. A production service would require stronger validation, expanded datasets, hardened integration, reliable storage, access control, and independent security testing.

11.3 Project Status Statement

Recommended submission wording: “SecureAPI Guard is an educational proof of concept for rule-based API request inspection and monitoring. Its current evaluation uses controlled synthetic inputs and local measurements. It is not presented as a production-ready WAF, and additional engineering and security testing are required before protecting real customer APIs.”

12. Conclusion and Future Work

12.1 Conclusion

SecureAPI Guard demonstrates the structure of a small API security inspection and monitoring application built with Node.js and Express. It brings together request processing, selected SQL injection and XSS pattern checks, middleware-based controls, metrics, browser-based interaction, and automated tests. The project provides a practical foundation for learning about API security and for demonstrating the distinction between detection, logging, and blocking.

The most important conclusion is that the current prototype should be evaluated according to what has actually been implemented and tested. Its limited rule-based detector and synthetic evaluation are useful for a proof of concept but are not sufficient to claim comprehensive attack prevention or production readiness.

12.2 Future Work

- Add secure authentication, role-based authorization, and verified customer data isolation.
- Introduce durable storage, migrations, backup, retention, and audit controls.
- Implement and test a safe gateway integration or installable middleware for explicitly authorized APIs.
- Improve detection rules and evaluate against a larger, balanced, documented dataset.
- Add regression tests for encoded inputs, evasions, false positives, and malformed requests.
- Harden backend resilience with centralized error handling, timeouts, health/readiness checks, graceful shutdown, and supervised recovery.

- Improve dashboard filters, charts, accessibility, loading states, and error handling using real backend data.
- Conduct an independent security review and controlled deployment test before any customer pilot.

References

Use the citation style required by your department. The following primary technical documentation is appropriate for the technologies discussed; consult the current versions when finalizing the report.

- Node.js. (n.d.). Node.js Documentation. https://nodejs.org/docs/Express.js. (n.d.). Express — Node.js web application framework. https://expressjs.com/
- OWASP Foundation. (n.d.). OWASP Top 10:2021 — A03:2021 Injection. https://owasp.org/Top10/A03_2021-Injection/
- OWASP Foundation. (n.d.). Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP Foundation. (n.d.). REST Security Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html
- Helmet. (n.d.). Helmet documentation. https://helmetjs.github.io/
- express-rate-limit. (n.d.). express-rate-limit documentation. https://express-rate-limit.mintlify.app/

Appendix A — Demonstration Scenarios

A.1 Normal Request

Purpose: demonstrate a benign request through the test endpoint. Record the endpoint, request payload, HTTP status, response body, and dashboard changes. Confirm that the

system does not label the request suspicious unless the configured rules justify the result.

A.2 Suspicious Pattern

Purpose: submit a controlled SQLi-like or XSS-like input to the project's own local test endpoint. Record whether the detector flags it, which rule or category is reported, whether an event is stored, and whether the request is actually blocked. Do not use a third-party target.

A.3 Error Resilience

Purpose: send malformed input or call an invalid route. Confirm that the server returns a controlled error and continues handling subsequent valid requests. Record logs and test output.

A.4 Dashboard Verification

Purpose: compare the dashboard's totals with backend responses and recorded events after a known sequence of test requests. The values should reconcile according to documented metric definitions.

Appendix B — Submission Verification Checklist

- Replace all cover-page placeholders with correct academic details.
- Confirm project name, folder structure, endpoint paths, and dependency versions from the final code.
- Run `npm.cmd test` and update all test results.
- Rerun detection evaluation and performance benchmark; retain raw outputs.
- Verify whether each described feature is implemented before marking it complete.
- Remove sample or historical numbers that cannot be reproduced.
- Inspect `.env.example`, source files, logs, and Git history for exposed secrets.

- Verify repository visibility and ensure no credentials or private customer data are committed.
- Add screenshots of the actual landing page, testing console, dashboard, and test output if required by the department.
- Add institution-required declaration, acknowledgement, dedication, supervisor approval, and formatting pages if applicable.
- Check grammar, page numbering, headings, figure/table numbering, and citation style.
- Have the advisor review the final version before submission.

Final Submission Note

This document is a polished draft built from the project details available at preparation time. It deliberately distinguishes previously recorded results from results that must be reproduced. The student is responsible for confirming the final source code, updating the test tables, adding real screenshots, and applying institutional submission requirements before submitting this report.